Abdelmalek Essaadi University
Faculty of Sciences and
Techniques of Tangier

Project Report

Adaptation of ANA to the TSP

Edge-Memory ANA Version

Prepared by: Youssef Khaloufi
Supervised by: Prof. Khalid Jebari

Academic year 2025–2026

Abstract

This report presents an adaptation of the Ant Nesting Algorithm (ANA) to the traveling salesman problem, known as the TSP. The original ANA algorithm is a continuous metaheuristic inspired by the behavior of *Leptothorax* ants during nest construction. In its original form, each ant represents a search agent whose position is updated using a deposit weight computed from the current position, the previous position, the best known position, and the associated fitness values.

The TSP is a discrete problem based on permutations. A solution is not a real-valued vector, but an order in which the cities are visited. For this reason, the original ANA cannot be applied directly. The work carried out therefore consists of preserving the general spirit of ANA while replacing the continuous operations with operations adapted to TSP tours.

The developed version, called Edge-Memory ANA, uses an edge memory in order to guide the search toward connections that often appear in good solutions. It also introduces two specific treatments: an escape strategy when the agent is already on the global best tour, and a mini-reconstruction when the agent repeats its previous tour. Finally, each new best solution is improved by a local search based on 2-opt and 3-opt.

Keywords: metaheuristics, ANA, TSP, permutations, edge memory, 2-opt, 3-opt.

Contents

Chapter 1

Introduction

1.1 General context

Optimization problems appear in many fields: transportation, planning, scheduling, network design, logistics, and engineering. In these problems, the objective is to choose the best possible solution according to a given criterion, for example minimizing a distance, a cost, or a time.

When the search space becomes very large, exhaustive exploration becomes unrealistic. Metaheuristics then form a useful family of methods. They do not always guarantee the global optimum, but they often make it possible to find good solutions in a reasonable amount of time.

1.2 Project objective

The objective of this project is to adapt the ANA algorithm to the traveling salesman problem. The goal is not to build a general metaheuristic library, but to produce a clear, simple, and functional version of ANA for a discrete problem based on permutations.

The work follows three main ideas:

- understand the mechanism of the original ANA;
- identify what cannot be used directly for the TSP;
- propose equivalent discrete operators while preserving the spirit of the algorithm.

1.3 Report organization

The report is organized as follows. Chapter 2 recalls the original ANA algorithm. Chapter 3 presents the TSP. Chapter 4 explains the proposed adaptation. Chapter 5 describes the functioning of the Edge-Memory ANA version. Chapter 6 presents the code organization and the experimental protocol. The final chapter concludes the work.

Chapter 2

Summary of the Original ANA Algorithm

2.1 Natural inspiration

The ANA algorithm is inspired by the behavior of *Leptothorax* ants during nest construction. Worker ants collect grains and deposit them around the queen and the brood. Gradually, the areas where deposits are most important become more attractive, which leads to the formation of a wall around the nest.

In the algorithm, this idea is transformed into an optimization mechanism. Each artificial ant represents a candidate solution. The movements of the ants correspond to the exploration of new solutions. The best known position plays the role of an attractive area.

2.2 Entities of ANA

As in the original article, natural elements are associated with algorithmic elements. The following table summarizes this correspondence.

Natural element	**Algorithmic element**
Worker ant	Search agent
Deposit position	Candidate solution
Quality of a position	Objective function / fitness
Best deposit area	Best solution found
Old position encountered	Previous position of the agent
Deposit weight	Intensity and direction of movement

2.3 General modeling

In the original ANA, a solution is represented by a continuous position $X_{t,i}$. At each iteration, the position of an agent is updated according to:

$$X_{t+1,i} = X_{t,i} + \Delta X_{t+1,i} \tag{2.1}$$

where $\Delta X_{t+1,i}$ represents the change in position. In the general case, this change is computed by:

$$\Delta X_{t+1,i} = dw \times (X_{\text{best}} - X_{t,i}) \tag{2.2}$$

The term dw is the deposit weight. It depends on a random number r and on a ratio between two tendencies: the current tendency T and the previous tendency $T_{previous}$. The idea is to compare the current situation of the agent with its previous situation, taking into account both the distance to the best solution and the fitness difference.

For a minimization problem, the principle can be summarized by:

$$dw = r \times \frac{T}{T_{previous}} \tag{2.3}$$

This mechanism makes it possible to combine exploration and exploitation:

- exploitation comes from the attraction toward the best solution;
- exploration comes from the random factor r and the possible change of direction.

2.4 Special cases of ANA

The original ANA also distinguishes two special situations.

First case: the agent is already on the best solution. In this case, the direction toward the best solution is zero. The algorithm then uses a form of random movement to prevent the agent from remaining stuck.

Second case: the current position is equal to the previous position. In this situation, the agent does not have real movement information. The algorithm therefore forces a new direction toward the best solution.

These two cases are important in our adaptation because they also correspond to possible blockages in a permutation space.

2.5 Operating mechanism

The general functioning of the original ANA can be summarized simply:

```
Initialize a population of artificial ants.
Initialize the previous position of each ant.
While the number of iterations has not been reached:
```

```
    Find the current best solution.
    For each ant:
        Generate a random number r.
        If the ant is on the best position:
            apply special case A.
        Else if current position = previous position:
            apply special case B.
        Else:
            compute T, Tprevious, dw, and the displacement.
        Generate a new solution.
        Accept the solution if it improves the fitness.
End.
```

Chapter 3

The Traveling Salesman Problem

3.1 Definition

The traveling salesman problem, or TSP, consists of finding the shortest tour that visits each city exactly once and returns to the starting city.

Formally, for a set of cities $V = \{0, 1, \ldots, n-1\}$ and a distance matrix $d(i, j)$, the goal is to find a permutation π that minimizes:

$$F(\pi) = \sum_{k=0}^{n-1} d(\pi_k, \pi_{k+1}) \tag{3.1}$$

with $\pi_n = \pi_0$.

3.2 Representation of a solution

In this project, a solution is represented by a permutation. For example:

```
[0, 3, 2, 5, 1, 4]
```

means that the tour starts at city 0, then visits 3, 2, 5, 1, and 4, before returning to 0.

To avoid several equivalent representations of the same tour, city 0 is always placed in the first position. After each operation, the tour is normalized in order to preserve this rule.

3.3 Why the original ANA is not sufficient

The original ANA manipulates continuous positions. An update such as:

$$X_{t+1} = X_t + \Delta X \tag{3.2}$$

is natural when X is a real-valued vector. But for the TSP, a solution must remain a valid permutation. It is not possible to directly add a vector to a tour.

The main challenge of the project is therefore the following:

How can the logic of ANA be preserved while replacing continuous movement with valid operators on permutations?

Chapter 4

Proposed Adaptation: Edge-Memory ANA

4.1 General idea

The proposed version preserves the structure of ANA: population of agents, global best solution, previous position, positive movement, negative movement, and special cases. The main difference is that continuous positions are replaced by TSP tours.

The project introduces an edge memory. An edge corresponds to a connection between two cities. In the TSP, the quality of a tour depends strongly on the edges it contains. It is therefore more natural to guide the search by edges than by the positions of cities in the permutation.

4.2 Edge memory

The edge memory is a symmetric matrix M of size $n \times n$. Each value M_{ij} indicates the learned interest for the undirected edge (i, j).

Initially, all edges have the same memory value. At each iteration:

- the memory evaporates slightly;
- the edges of the global best tour receive a deposit;
- the edges of accepted solutions that are close to the best may also receive a smaller deposit.

Evaporation is given by:

$$M_{ij} \leftarrow (1 - \lambda) M_{ij} \tag{4.1}$$

where λ is the evaporation rate.

When a good tour contains the edge (i, j), the following update is applied:

$$M_{ij} \leftarrow \min(M_{ij} + \delta, M_{max}) \tag{4.2}$$

This memory plays a role close to the collective experience of the population.

4.3 Ranking of memorized edges

To avoid scanning all edges at each movement, the edges are ranked in a cache. This cache is marked as dirty when evaporation or a deposit modifies the memory. It is rebuilt only when necessary.

The score of an edge can be weighted by distance:

$$score(i, j) = \frac{M_{ij}}{d(i, j)} \tag{4.3}$$

Thus, an edge is interesting if it has a high memory value and a short distance.

4.4 Positive movement

In the original ANA, a positive movement brings the agent closer to the best solution. In our version, this principle is translated as follows:

A positive movement tries to insert into the current tour strongly memorized edges that are not yet present.

To insert an edge (a, b) into a tour, the algorithm uses a segment inversion. This operation always preserves a valid permutation.

4.5 Negative movement

When the deposit weight is negative, the algorithm performs a random inversion of a segment of the tour. This movement plays the role of exploration. It makes it possible to leave a search area without breaking the validity of the permutation.

4.6 Case A: the agent is on the global best tour

In the original ANA, when the agent is already on the best position, the direction toward the best solution is zero. In our version, this situation means that the current tour is exactly the global best tour.

To avoid blockage, the Edge-Memory ANA version applies an adaptive escape:

- if stagnation is low, the algorithm tries to insert a memorized edge;
- if stagnation increases, it tries more insertions;
- if no useful insertion is possible, it applies a random inversion.

4.7 Case B: the current tour is equal to the previous tour

When the current tour is identical to the previous tour, the agent has not really progressed. In this case, the proposed version uses a mini-reconstruction.

The mini-reconstruction inserts several memorized edges into the current tour. If it fails, the algorithm uses a small random inversion. This operation gives the agent a new direction.

4.8 Acceptance by simulated annealing

The original ANA mainly accepts improvements. In our version, a simulated annealing rule is added in order to sometimes allow the acceptance of a worse solution. This helps avoid certain local optima.

If a candidate solution is better, it is accepted. Otherwise, it can be accepted with the probability:

$$p = \exp\left(-\frac{\Delta f / f}{T}\right) \tag{4.4}$$

where Δf is the deterioration of the fitness, f is the current fitness, and T is the current temperature.

4.9 Local improvement

Each time a new global best tour is found, it is improved by a local search:

1. apply 2-opt until no further improvement is possible;
2. try a strict 3-opt improvement;
3. if 3-opt improves, run 2-opt again;
4. repeat until stabilization.

This step is important because ANA builds good global directions, while 2-opt and 3-opt locally correct the structure of the tour.

Chapter 5

Complete Mechanism of the Final Version

5.1 Pseudo-code

The functioning of the final version can be summarized as follows:

```
Initialize the population of tours.
Compute the fitness of each tour.
Determine the global best tour.
Initialize the edge memory.
Deposit the edges of the best tour in the memory.

For each iteration:
    Evaporate the edge memory.
    Redeposit the edges of the global best tour.
    Compute the current temperature.

    For each agent:
        If the current tour is the global best:
            apply case A: adaptive escape.
        Else if current tour = previous tour:
            apply case B: mini-reconstruction.
        Else:
            compute the discrete ANA movement.
            if dw > 0: insert memorized edges.
            if dw < 0: apply a segment inversion.

        Compute the fitness of the candidate tour.
        Accept if it improves the current tour.
        Otherwise accept sometimes with simulated annealing.
```

```
        If the candidate is close to the best:
            deposit its edges in the memory.

        If the candidate improves the global best:
            apply 2-opt + 3-opt.
            update the global best.
            deposit its edges in the memory.

Return the best tour found.
```

5.2 What was changed compared with ANA

The following table summarizes the main changes.

Original ANA	**Edge-Memory ANA Version**
Continuous position X	TSP tour represented by a permutation
Addition of a displacement ΔX	Discrete operators: inversion and edge insertion
Direction toward X_{best}	Guidance by memorized edges
Continuous distance between positions	Indirect similarity distance through swaps and edge structure
Case agent = best	Adaptive escape by memory, then inversion
Case current = previous	Mini-reconstruction by memorized edges
Strict acceptance of improvements	Acceptance with simulated annealing for some deteriorations
No specific TSP local search	2-opt + 3-opt polishing on each new best solution

5.3 Main parameters

The most important parameters are:

- the population size;
- the maximum number of iterations;
- the parameter ρ, used to control the amplitude of certain movements;
- the memory evaporation rate;
- the deposit on the global best solution;
- the deposit on solutions close to the best;
- the initial and minimum temperature of simulated annealing.

Chapter 6

Project Implementation

6.1 Simplicity choices

The final project is not a large software platform. It is a simple and readable implementation of the final version of the algorithm. The objective is to make the code understandable by separating only the essential responsibilities.

6.2 Code structure

The final structure of the project is the following:

```
edge_memory_ana_tsp/
    main.py
    core/
        algorithm_flow.py
        operators.py
        local_search.py
        tsp_utils.py
    experiments/
        benchmarks.py
        runner.py
    reporting/
        results_writer.py
    data/
        wi29.tsp
        dj38.tsp
    results/
    tests/
        smoke_test.py
```

The most important file is:

`core/algorithm_flow.py`

It contains the general flow of the algorithm: initialization, iteration loop, candidate generation, acceptance, memory update, and return of the final result.

The statistics, CSV writing, and summary files are deliberately placed in the `reporting/` folder, in order not to mix the algorithm logic with the presentation of results.

6.3 Benchmarks used

The final version uses only the following benchmarks:

Benchmark	**Type**	**Known optimum**
square_4	synthetic	4.0
rectangle_6	synthetic	10.0
grid_9	synthetic	9.414213562373095
wi29	TSPLIB	27603
dj38	TSPLIB	6656

For the TSPLIB instances, Euclidean distances are rounded according to the EUC_2D convention.

6.4 Validation criteria

A run is considered successful when the best fitness found is equal to the known optimum. The relative gap is computed by:

$$Gap(\%) = 100 \times \frac{f_{best} - f_{opt}}{f_{opt}} \tag{6.1}$$

A gap of 0% means that the known optimum has been reached.

Chapter 7

Results and Discussion

7.1 Objective of the evaluation

The experimental evaluation does not try to prove that this adaptation outperforms specialized TSP solvers. The objective is more reasonable: to verify that the final algorithm is correct, stable on the selected instances, and capable of reaching the known optima with the parameters fixed in the project.

The results presented below are therefore summary results. The individual lines of each run are not necessary in the report: they are kept in the CSV files generated by the code, while the report presents only the useful indicators for judging the quality of the final version.

7.2 Final results on the selected benchmarks

The following table presents the retained results for the final Edge-Memory ANA version. For each benchmark, the number of runs, the known optimum, the obtained values, and the number of successes are indicated.

Benchmark	Runs	Optimum	Best	Average	Success	Avg. time
square_4	10	4.0	4.0	4.0	10/10	0.028 s
rectangle_6	10	10.0	10.0	10.0	10/10	0.099 s
grid_9	30	9.4142	9.4142	9.4142	30/30	1.061 s
wi29	10	27603	27603	27603	10/10	37.74 s
dj38	10	6656	6656	6656	10/10	38.96 s

Table 7.1: Final results retained for the Edge-Memory ANA version.

These results show that the final version reaches the known optimum on all benchmarks kept in the project. The three small synthetic benchmarks mainly serve to verify the validity of the implementation: correct permutation, city 0 fixed, correct tour cost,

and coherent behavior. The two TSPLIB instances, wi29 and dj38, confirm that the algorithm also works on real instances with integer distances.

7.3 Experimental progression of the method

During development, several versions were tested. The following table does not present all executions, but only the useful steps for understanding why the final version was retained.

Version	Main idea	Runs	Best	Average	Success
V5	Discrete adaptation with 2-opt and 3-opt polishing on new best solutions.	50	27603	28127.7	8/50
V9	Addition of edge memory and guidance by strongly memorized edges.	50	27603	27744.12	33/50
V11	Final version: edge memory, adaptive case A and case B by mini-reconstruction.	10	27603	27603	10/10

Table 7.2: Summary of the experimental progression on the wi29 instance.

This progression shows the interest of edge memory. The transition from V5 to V9 strongly improves robustness on wi29: the number of successes increases from 8/50 to 33/50 and the average gets closer to the optimum. Version V11 then adds the two blockage treatments inspired by the special cases of ANA: adaptive escape when the agent is already on the best tour, and mini-reconstruction when the current tour repeats the previous tour.

This table should be interpreted as a development synthesis, not as a complete statistical study. The numbers of runs are not identical across all rows. Its role is to show the improvement logic that led to the final version.

7.4 Result analysis

The obtained results confirm three important points.

First, the permutation representation is correct. All returned tours remain valid: each city appears exactly once and city 0 is preserved in the first position after normalization.

Second, edge memory is adapted to the TSP. In a tour problem, quality depends strongly on the connections between cities. Guiding movements by learned edges is therefore more natural than a direct positional displacement.

Third, 2-opt/3-opt polishing complements the global search well. The discrete ANA explores and proposes new tours, then local search corrects crossings and improves the details of the tour. This combination explains the ability of the final version to reach the retained optima.

7.5 Limitations

These results show that the project is functional and coherent, but they should not be overinterpreted. The TSP has much more powerful specialized solvers. The objective here is to study an adaptation of ANA to the TSP, not to replace exact methods or highly specialized heuristics such as Lin-Kernighan.

The main limitation concerns scaling. When the number of cities increases, 3-opt local search becomes costly. A possible continuation would therefore be to optimize computation time, reduce the cost of local search, or compare the method more formally with ACO, GA, SA, and other classical metaheuristics.

Chapter 8

Conclusion

This project made it possible to adapt the ANA algorithm, initially designed for continuous problems, to the discrete traveling salesman problem. The adaptation is based on a simple idea: replacing continuous movement with valid operators on permutations.

The final version, Edge-Memory ANA, preserves the essential principles of ANA: population of agents, best solution, previous position, guided movement, and special cases. It adds an edge memory, simulated annealing acceptance, and local 2-opt/3-opt polishing.

The final result is a simple, readable, and functional implementation. The project shows how a continuous metaheuristic can be transformed into a discrete method, provided that the structure of the problem being addressed is respected.

Perspectives

Possible improvements exist, notably:

- compare the algorithm more formally with classical methods such as ACO, GA, or SA;
- test other TSPLIB instances;
- study the influence of memory parameters;
- improve the speed of local search.

However, within the scope of this project, the main objective was to build a clear, clean, and understandable version of ANA for the TSP. This objective has been achieved.

Bibliography

[1] D. N. Hama Rashid, T. A. Rashid, S. Mirjalili, *ANA: Ant Nesting Algorithm for Optimizing Real-World Problems*, Mathematics, 2021.

[2] G. Reinelt, *TSPLIB - A Traveling Salesman Problem Library*, ORSA Journal on Computing, 1991.

[3] G. A. Croes, *A Method for Solving Traveling-Salesman Problems*, Operations Research, 1958.